Dokumentationskonzept und Designstudie

Modul: T4INF2003 – Software Engineering I
Kurs: TINF2024
Dozentin: Charlotte Chichlow
Semester: Frühjahr 2026

Projekt: Entwicklung eines Online Aufgaben-Planungssystems

Dokumentationskonzept und Designstudie **1**
1 Dokumentationskonzept **2**
1.1 Ziel der Dokumentation 2
Interne Dokumentation 2
Externe Dokumentation 2
1.2 Qualitätsstandards 2
1.3 Dokumentationsregeln 3
1.4 Teststrategie 4
1.6 Rollen und Verantwortlichkeiten 6
2 Designstudie **7**
2.1 Allgemeines 7
2.2 Produktübersicht 7
2.3 Architektur der Anwendung 8
2.4 Struktur der Komponenten 10
WorkProfile Entity 13
TaskDependency Entity 13
2.5 Frontend-Struktur 14
2.7 Scope der Designstudie - Etappen-Gliederung 14
2.8 Deployment & Release-Strategie 15

1 Dokumentationskonzept

1.1 Ziel der Dokumentation

Ziel der Dokumentation ist es, die Qualität, Wartbarkeit und Verständlichkeit der entwickelten Software sicherzustellen.
Die Dokumentation soll es auch Personen ermöglichen, die bisher nicht am Projekt beteiligt waren, sich schnell in das System und dessen Struktur einzuarbeiten.

Die Dokumentation besteht aus zwei Teilen:

Interne Dokumentation

Die interne Dokumentation befindet sich direkt im Quellcode und umfasst:

- Kommentare zu Klassen und Methoden
- Beschreibung wichtiger Parameter und Rückgabewerte
- kurze Erläuterung komplexerer Programmlogik

Externe Dokumentation

Die externe Dokumentation beschreibt das System auf höherer Ebene und enthält:

- Designbeschreibung der Software
- Beschreibung der Systemarchitektur
- Überblick über Komponenten und Module
- kurze Anleitung zur Nutzung der Anwendung

1.2 Qualitätsstandards

Zur Sicherstellung einer guten Softwarequalität werden folgende Prinzipien eingehalten.

Allgemeine Standards:

- Einheitlich und lesbar sein
- Konsistente Namenskonventionen verwenden
- Modular aufgebaut sein
- Ausreichend und verständlich kommentiert sein
- Immer die neuesten Sprachfeatures von C# verwenden
- Keine unnötig überkomplex Logik enthalten
- Verständliche Klassen- und Methodennamen enthalten
- Englische Variablen, Methoden und Klassennamen enthalten
- In kleine, übersichtliche Funktionen aufgeteilt sein
- Redundante Logik vermeiden
- Eine klare Trennung zwischen Benutzeroberfläche, Geschäftslogik und Datenzugriff aufweisen

C#-spezifische Standards:

- Nur Exceptions catchen, die man bearbeiten kann
- Bei Exceptions nur verständliche Fehlermeldungen ausgeben
- Async/await bei I/O-Operationen verwenden
- LINQ bei Collection Manipulation verwenden
- Language Keywords anstatt runtime types verwenden (z.B. string statt System.String)
- Möglichst Dependency Injection verwenden

Namenskonventionen

- Backend (C#): PascalCase für Klassennamen, Methodennamen und Dateinamen
  - Beispiel: UserService.cs, CreateTaskAsync(), TaskRepository
- Frontend (React): kebab-case für Komponentendateinamen, camelCase für Variablen
  - Beispiel: task-list.tsx, handleTaskSubmit()

1.3 Dokumentationsregeln

Jede zentrale Klasse enthält eine kurze Beschreibung ihrer Aufgabe.

Zur Dokumentation des Quellcodes werden Dokumentationskommentare verwendet. Diese sind XML-Text, den man vor Methoden schreibt und diese damit dann beschreibt. Mit dem Dokumentationsgenerator wird dann die Dokumentationsdatei generiert. Der Text in den Dokumentationskommentaren muss gemäß der XML Regeln wohlgeformt sein

Wichtige Methoden enthalten:

- Alle wichtigen Methoden enthalten:
- <summary>: Beschreibung der Funktion
- <param>: Beschreibung aller Parameter
- <returns>: Beschreibung des Rückgabewerts
- <exception>: Auflistung möglicher Exceptions

Weitere Details zu Dokumentationskommentaren entnimmt man hier: https://learn.microsoft.com/de-de/dotnet/csharp/language-reference/language-specification/documentation-comments

Neben der Quellcode-Dokumentation wird eine Designbeschreibung erstellt, die folgende Aspekte erklärt:

- Gesamtarchitektur der Anwendung
  - Beschreibt den grundlegenden Aufbau der Software
- Struktur der wichtigsten Komponenten

    - Beschreibt die wichtigsten Komponenten die in der Architektur verwendet werden
- grundlegende Designentscheidungen
    - Begründet welche Entscheidungen aus welchem Grund getroffen wurden

1.4 Teststrategie

Zur Überprüfung der Funktionalität werden Unit-Tests eingesetzt.

Ziele der Tests:

- Überprüfung einzelner Funktionen (Unit Tests)
- Überprüfung wichtiger Programmpfade
- Überprüfung der Zusammenarbeit zwischen Komponenten (Integration Tests)
- Frühzeitiges Erkennen von Fehlern
- Sicherung der Funktionalität bei Refactoring
- Test-Ebenen

1. Unit Tests (Backend: MSTest)

- Testen einzelner Funktionen in Isolation
- Mock-Objekte für externe Dependencies verwenden
- Testbeispiele für alle wichtigen Code-Branches
- Ausführung: Nach der Erstellung eines Moduls

Beispiel: TaskServiceTests.cs, UserRepositoryTests.cs

2. Integration Tests (Backend: MSTest)

- Testen der Zusammenarbeit zwischen Komponenten
- Überprüfung der Datenverwabe zwischen Services
- Überprüfung der Datenbank-Operationen

Ausführung: Vor jedem Pull Request, wenn Komponenten verbunden werden

3. UI Tests (Frontend: Playwright)

- Testen der Benutzeroberfläche
- Überprüfung von User-Workflows (z.B. Task erstellen → anzeigen → bearbeiten)
- Überprüfung von Formularen und Validierungen

Ausführung: Nach der Erstellung neuer UI-Komponenten

4. System Tests

- Testen der gesamten Anwendung
- Überprüfung aller Anforderungen

Ausführung: Vor dem Mergen auf den main-Branch

5. Benutzer-Akzeptanz-Tests

- Testen aus der Perspektive des End Nutzers
- Durchspielen realistischer Nutzungsszenarien

Ausführung: Nach Fertigstellung neuer Features (vor Release)

Automatisierung:

- Unit und Integration Tests laufen automatisch bei jedem Commit
- UI Tests laufen vor jedem Merge zu main
- Tests müssen grün sein, bevor ein PR gemerged werden kann
- Zielwert für Code-Coverage: mindestens 80%
- Fehlgeschlagene Tests blockieren den Merge zum main-Branch

Workflow:

- Developer Push → GitHub Actions aktiviert
- Unit Tests (MSTest) → Muss bestehen
- Integration Tests (MSTest) → Muss bestehen
- UI Tests (Playwright) → Muss bestehen
- Code Quality Check → Muss bestehen
- Coverage Report → Muss >= 80% sein

1.5 Werkzeuge zur Unterstützung der Entwicklung

Bereich	Werkzeug	Zweck
Versionskontrolle	Git / GitHub	Source Code Verwaltung, Branches, PRs
Issue Tracking	Jira	Task-Management, Bugtracking, Sprint-Planung
Continuous Integration	GitHub Actions	Automatisierte Tests, Builds, Deployment
Dokumentation	Markdown / PlantUML	technische Dokumentation und Diagramme
Backend Tests	MSTest	Unit und Integration Testing Framework

UI Tests	Playwright	End-to-End Testing des Frontends
Code Review Tool	GitHub PRs	Qualitätssicherung durch Peer Review

Teamabsprachen zur Werkzeugnutzung

GitHub Workflow:

- Feature Branches folgen dem Pattern: feature/task-name
- Bug Branches: bugfix/bug-name
- Hotfix Branches: hotfix/bug-name
- Mindestens 1 Code Review vor Merge
- Alle Checks (Tests, Coverage) müssen grün sein

Jira Workflow:

- Jedes Issue hat einen Story Point
- Jedes Issue wird einem Sprint zugewiesen
- Status: To Do → In Progress → In Review → Done

Code Quality:

- GitHub Actions prüft Tests automatisch
- Code Coverage muss >= 80% sein
- Linting muss bestanden sein

1.6 Rollen und Verantwortlichkeiten

Dokumentations-Verantwortlicher Aufgaben:

- Sichert Einhaltung der Dokumentationsstandards
- Kontrolliert Quellcode-Kommentare vor Merge (im Code Review)
- Verwaltet externe Designdokumentation und hält sie aktuell
- Aktualisiert Designbeschreibung bei größeren Änderungen
- Erstellt und pflegt das Architektur- und Datenmodell-Diagramme
- Dokumentiert neue Design Patterns oder Architektur-Entscheidungen

Test-Verantwortlicher Aufgaben:

- Plant und pflegt die Teststrategie
- Schreibt und pflegt Test-Cases für kritische Funktionen
- Führt Code Reviews durch (Test-Aspekte prüfen)
- Verwaltet Test-Dokumentation
- Überwacht Code Coverage und arbeitet an Verbesserungen
- Koordiniert UI Tests und verwaltet Playwright-Test-Suite

Alle Entwickler Aufgaben:

- Schreiben aussagekräftige XML-Dokumentationskommentare
- Testen ihren eigenen Code mit Unit Tests
- Schreiben Tests BEVOR sie Features implementieren (TDD-Ansatz wenn möglich)
- Befolgen die definierten Naming Conventions
- Führen lokale Tests aus vor dem Push

- Aktualisieren Dokumentation bei größeren Changes

Tech Assist/ Projekt Manager Aufgaben:

- Überwacht Einhaltung der Qualitätsstandards
- Leitet Code Reviews
- Entscheidet über Einsatz neuer Technologien
- Plant Testabdeckung und Releases

2 Designstudie

2.1 Allgemeines

Die Designstudie dient dazu, erste praktische Erfahrungen mit der technischen Umsetzung eines Online Aufgaben-Planungssystems zu sammeln. Dabei wird ein Prototyp erstellt, der grundlegende Funktionen des Systems demonstriert.

Ziele der Designstudie:

- Validierung der gewählten Technologien (React, C#, SQL Server)
- Prototypische Umsetzung zentraler Geschäftslogik
- Sammlung von Erfahrungen mit der geplanten Architektur
- Identifikation von Risiken und Herausforderungen früh erkennen
- Grundlage für das Pflichtenheft und nachfolgende Entwicklung

Der Prototyp dient ausschließlich zur Erprobung zentraler Konzepte und stellt keine vollständige Produktversion dar

2.2 Produktübersicht

Das System ist eine Webanwendung zur Planung und Verwaltung von Aufgaben innerhalb einer Organisation. Es richtet sich an Organisationen und Teams, welche Aufgaben planen und koordinieren möchten. Ziel ist es, Aufgaben verwalten zu können und sinnvolle Arbeitspläne zu generieren.

Benutzer können:

- Sich registrieren und anmelden
- Aufgaben erstellen, bearbeiten und löschen
- Aufgaben als erledigt markieren
- Ein persönliches Arbeitsprofil konfigurieren
- Einen Arbeitsplan automatisch generieren lassen
- Ihre Aufgaben in einer strukturierten Übersicht anzeigen

Das System unterstützt außerdem:

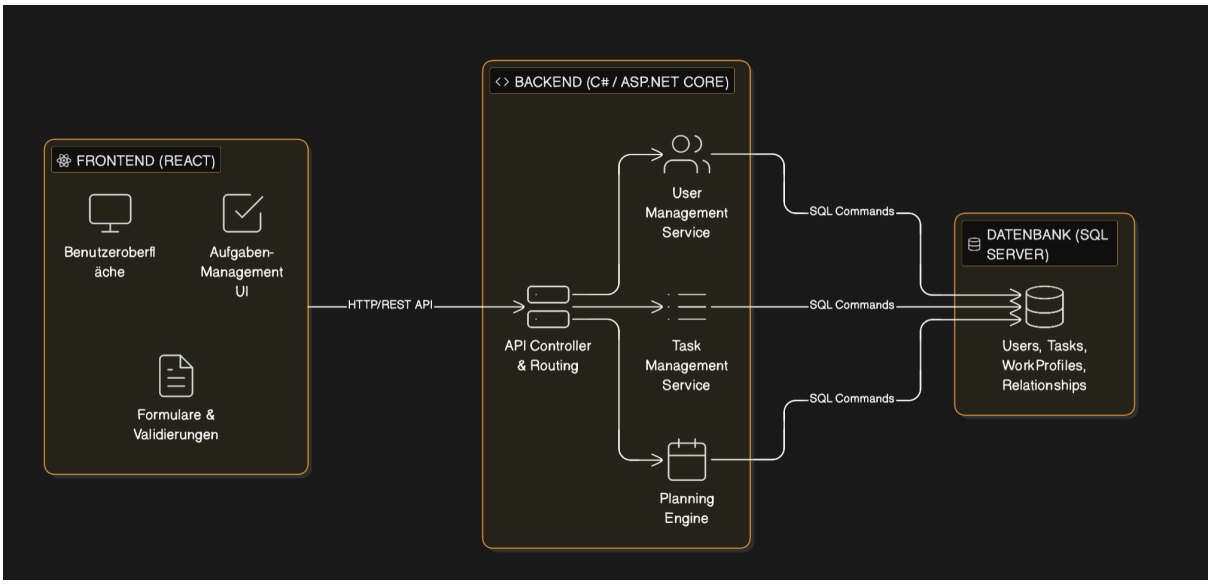
- Prioritäten (High, Medium, Low)
- Deadlines für Aufgaben

- Aufgabenabhängigkeiten (Aufgabe A muss vor B erledigt sein)
- Individuelle Arbeitsprofile mit Arbeitszeit-Einstellungen
- Automatische Arbeitsplan-Generierung basierend auf Aufgabeneigenschaften

2.3 Architektur der Anwendung

Die Anwendung verwendet eine **Client-Server-Architektur**.

Durch diese Architektur ist es möglich, Frontend von Backend klar zu trennen und beide unabhängig voneinander zu erweitern. Gleichzeitig besteht dadurch noch die möglichkeit dass mehrere Clients auf denselben Server zugreifen können.



Begründung der Architektur-Entscheidung

Client-Server-Architektur wurde gewählt, weil**:**

- Separation of Concerns: Frontend und Backend können unabhängig entwickelt und gewartet werden
- Skalierbarkeit: Mehrere Frontend-Clients können denselben Backend-Server nutzen
- Sicherheit: Authentifizierung und Autorisierung zentral auf dem Server
- Wartbarkeit: Änderungen am Backend beeinflussen nicht automatisch das Frontend
- Testing: Backend und Frontend können isoliert getestet werden

Frontend-Schicht: React

Technologie: React 18+

Aufgaben:

- Benutzeroberfläche darstellen

- Aufgaben in verschiedenen Views anzeigen (Liste, Kalender)
- Formulare für Task-Erstellung und -Bearbeitung bereitstellen
- Benutzerinteraktion ermöglichen und Feedback geben
- State Management für UI-Zustand

Warum React?

- Große Community & umfangreiche Dokumentation
- Wiederverwendbare Komponenten (DRY-Prinzip)
- Sehr gute Developer Experience und Tools
- Gutes Rendering-Performance durch Virtual DOM
- Einfaches State Management mit Hooks

Backend-Schicht: C# / ASP.NET Core

Technologie: C# 11+, ASP.NET Core 8+

Aufgaben:

- Benutzerverwaltung (Registrierung, Authentifizierung, Login)
- Aufgabenverwaltung (CRUD-Operationen)
- Generierung von Arbeitsplänen (Planungs-Algorithmus)
- Zugriff auf Datenbank mit Entity Framework
- REST-API bereitstellen
- Input-Validierung und Error Handling

Warum C#?

- Starke Typsicherheit & Compiler-Fehlerprüfung
- Modernes async/await-Modell für nicht-blockierende I/O-Operationen
- Entity Framework ORM für einfachen Datenbankzugriff
- Gute Performance und Skalierbarkeit
- LINQ für elegante Datenverarbeitung
- Built-in Dependency Injection in ASP.NET Core

Datenbank-Schicht: SQL Server

Technologie: SQL Server 2019+

Aufgaben:

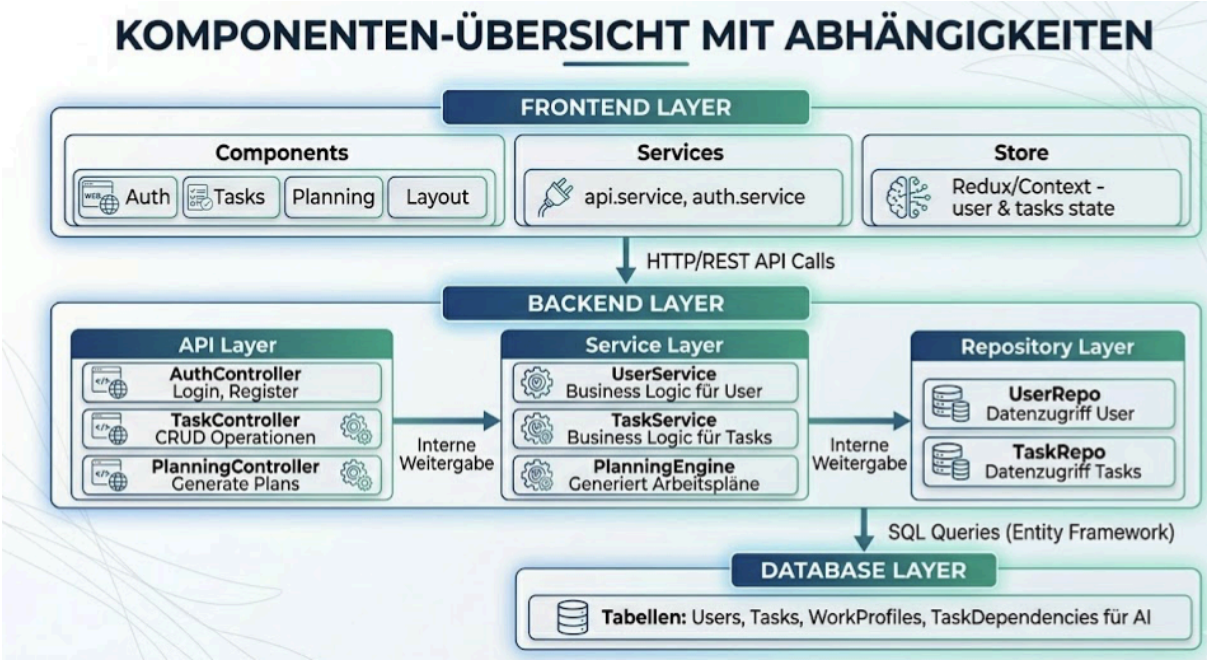
- Persistente Speicherung von Benutzerdaten
- Speicherung von Aufgaben und deren Eigenschaften
- Speicherung von Arbeitsprofilen
- Verwaltung von Beziehungen zwischen Entities
- Indizes für Performance-Optimierung

Warum SQL Server?

- Zuverlässig und produktionsreif
- Gute Integration mit C# / Entity Framework
- ACID-Compliance für Datenkonsistenz
- Skalierbarkeit für größere DatenmengenTechnologie: React

2.4 Struktur der Komponenten

Um die Anwendung übersichtlich und wartbar zu halten wird das System in die folgenden Einzelnen Komponenten unterteilt



User Management Component

Verantwortung:

- Benutzerverwaltung (Registrierung, Login, Profil)
- Authentifizierung & Autorisierung
- Passwort-Hashing & Sicherheit

Enthaltene Services:

- UserService: Geschäftslogik für Benutzerverwaltung
- UserRepository: Datenbankzugriff für User-Entities
- AuthenticationService: JWT Token-Handling

Abhängigkeiten:

- Task Management (zur Verknüpfung von Aufgaben zu Benutzern)
- Datenbankschicht

Task Management Component

Verantwortung:

- Aufgabenverwaltung (Erstellen, Bearbeiten, Löschen, Anzeigen)
- Aufgaben-Validierung
- Aufgaben-Filtering und Sortierung
  Enthaltene Services:
- TaskService: Geschäftslogik für Task-Verwaltung
- TaskRepository: Datenbankzugriff für Task-Entities
- TaskValidator: Validierung von Task-Daten

Abhängigkeiten:

- User Management (Authentifizierung)
- Planning Engine (bei Änderungen können Pläne neuberechnet werden)
- Datenbankschicht

Planning Engine Component

Verantwortung:

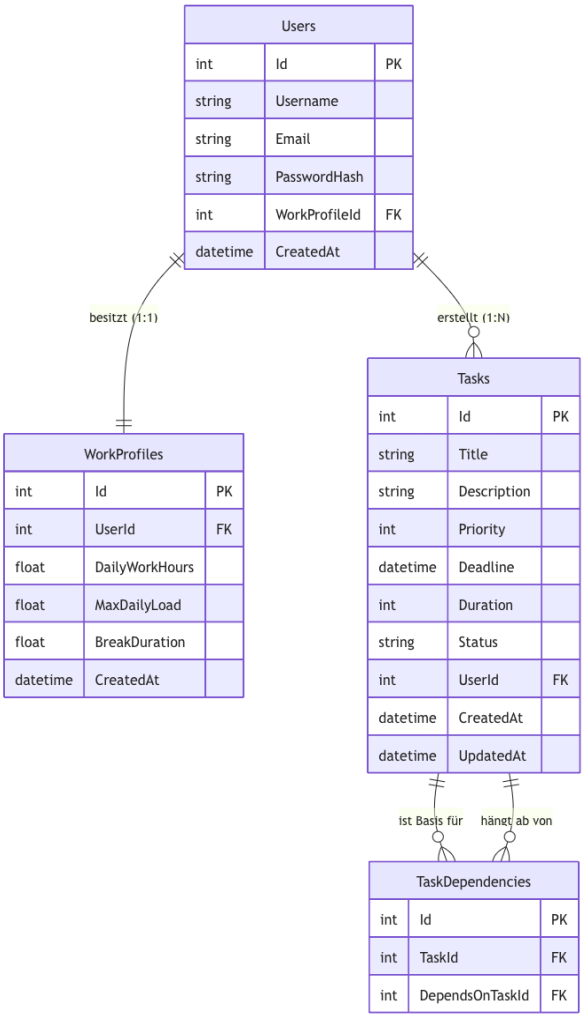
- Automatische Generierung von Arbeitsplänen
- Algorithmus zur Aufgaben-Schedulierung
- Berücksichtigung aller Constraints (Deadline, Priorität, etc.)

Enthaltene Services:

- PlanningService: Orchestriert den Planungsprozess
- SchedulingAlgorithm: Kernalgorithmus für Aufgaben-Schedulierung
- WorkloadCalculator: Berechnet Arbeitsbelastung

Abhängigkeiten:

- Task Management (Aufgabendaten lesen)
- User Management (Benutzerprofildaten lesen)
- Datenbankschicht



User Entity

```
public class User
{
    public Guid Id { get; set; }
    public string Username { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; }
    public Guid WorkProfileId { get; set; }
    public DateTime CreatedAt { get; set; }

    // Navigation properties
    public WorkProfile WorkProfile { get; set; }
    public ICollection<Task> Tasks { get; set; }
}
```

Task Entity

```
public class Task
{
```

```
    public Guid Id { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public PriorityLevel Priority { get; set; } // High, Medium, Low
    public DateTime Deadline { get; set; }
    public float Duration { get; set; } // in Stunden
    public TaskStatus Status { get; set; } // Open, InProgress, Done
    public Guid UserId { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }

    // Navigation properties
    public User User { get; set; }
    public ICollection<TaskDependency> DependenciesAsMain { get; set; }
    public ICollection<TaskDependency> DependenciesAsDependent { get; set; }
}
```

WorkProfile Entity

```
{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public int DailyWorkHours { get; set; } // z.B. 8
    public float MaxDailyLoad { get; set; } // z.B. 8.0 (Stunden)
    public TimeSpan BreakDuration { get; set; } // z.B. 1 Stunde

    // Navigation properties
    public User User { get; set; }
}
```

TaskDependency Entity

```
public class TaskDependency
{
    public Guid Id { get; set; }
    public Guid TaskId { get; set; }
    public Guid DependsOnTaskId { get; set; }

    // Navigation properties
    public Task Task { get; set; }
    public Task DependsOnTask { get; set; }
}
```

2.5 Frontend-Struktur

Die React-Anwendung wird in folgenden Hauptbereichen organisiert:

Components:

- Authentication (Login, Register, Logout)
- Task Management (TaskList, TaskForm, TaskDetail)
- Planning (GeneratePlanButton, PlanningEngine)
- Layout (Navbar, Sidebar, Layout-Wrapper)

Services:

- API Service (HTTP-Anfragen zum Backend)
- Authentication Service (JWT Token Handling)

State Management:

- Redux Toolkit oder Context API für:
- User State (Authentifizierung, Profil)
- Task State (Aufgabenliste, Filter)
- Plan State (Generierter Arbeitsplan)

  Diese modulare Struktur ermöglicht:
- Wiederverwendbarkeit von Komponenten
- Einfacherer Testing
- Bessere Wartbarkeit
- Klare Separation of Concerns

2.6 Verwendete Design Patterns

Wir verwenden folgende Design Patterns in unserer Anwendung, um die Qualität und Wartbarkeit des Codes zu gewährleisten:

- Repository Pattern: Zentralisiert den Zugriff auf die Datenbank, um Wartbarkeit und Testbarkeit zu erhöhen.
- Dependency Injection (DI): Löst feste Abhängigkeiten auf und macht Code modular und flexibler.
- Service Layer Pattern: Kapselt die Geschäftslogik und ermöglicht einfaches Testen und Erweitern.
- Factory Pattern: Komplexe Objekt-Erstellung wird zentralisiert und erleichtert die Erweiterung.
- Strategy Pattern: Ermöglicht die Implementierung verschiedener Planungs-Algorithmen und macht die Logik erweiterbar

2.7 Scope der Designstudie - Etappen-Gliederung

Die Designstudie konzentriert sich ausschließlich auf die Implementierung der Funktionen, die für die **Etappe 1 (Minimum Viable Product – MVP)**aus dem Lastenheft definiert wurden. Funktionen, die für Etappe 2 und 3 vorgesehen sind, werden hier als **No-Go** festgelegt, um den Fokus und den zeitlichen Rahmen des Semesters (Randbedingung) einzuhalten.

Die folgende Tabelle zeigt die konkrete Go/No-Go-Entscheidung für die funktionalen Anforderungen (FXX) des MVP:

Funktion (FXX)	Lastenheft-Ziel	Go/No-Go Entscheidung	Begründung für MVP (Etappe 1)
F20 Registrierung & Login	Benutzerverwaltung (Muss-Ziel)	**Go**	Essenzielle Basis-Funktion zur Nutzeridentifikation und Zugriffssicherung.
F50 Arbeitsprofil konfigurieren	Eingabe der Belastbarkeit (Muss-Ziel)	**Go**	Notwendig, um die Eingangsdaten für den Planungsalgorithmus zu liefern.
F10 Aufgabe konfigurieren	Aufgabenverwaltung (CRUD) (Muss-Ziel)	**Go**	Die zentrale Funktion zur Datenerfassung für die Planung.
F40 Arbeitsplan generieren	Algorithmus zur Planung (Muss-Ziel)	**Go**	Die Kernfunktionalität des Produkts.
F30 Organisierende einladen	Multi-User-Erweiterung (Kann-Ziel, Etappe 2)	**No-Go**	Führt komplexe Berechtigungs- und Organisationsstrukturen ein, die außerhalb des MVP-Scopes liegen.
F60 Transparente Planung	Nachvollziehbarkeit des Algorithmus (Kann-Ziel, Etappe 3)	**No-Go**	Die Implementierung der Planungslogik hat Priorität; die erweiterte Visualisierung und Erklärung der Logik wird verschoben.

2.8 Deployment & Release-Strategie

Die Bereitstellung der Anwendung erfolgt über eine automatisierte Pipeline, um eine schnelle, zuverlässige und reproduzierbare Veröffentlichung zu gewährleisten.

Bereich	Strategie / Werkzeug
Deployment-Ziel	Die gesamte Anwendung (Frontend und Backend) wird in **Docker-Container** gepackt. Das Deployment erfolgt auf einem stabilen Webhosting-Service (z. B. einem Virtual Private Server, alternativ Azure oder AWS), der die Ausführung von Containern unterstützt.
CI/CD-Pipeline	Zur Automatisierung wird **GitHub Actions** verwendet (siehe 1.5). Die Pipeline umfasst: Code-Build → Ausführung aller Unit- und Integration-Tests → Erstellung des Docker-Images → Push in ein Container Registry → Automatisches Deployment (CD) bei erfolgreichem Build.
Release-Prozess	Das Deployment wird automatisch nach erfolgreichem Merge in den ```main```-Branch ausgelöst. Alle Tests müssen den Zustand „Grün“ haben, und die Code Coverage muss mindestens 80% betragen (siehe 1.4).
Versioning	Es wird **Semantic Versioning**(```MAJOR.MINOR.PATCH```) verwendet. Änderungen am Frontend, Backend oder den Datenbank-Migrationen werden durch eine Erhöhung der Versionsnummer kenntlich gemacht.
Rollback-Strategie	Im Falle eines kritischen Fehlers nach dem Deployment (z. B. durch einen Systemtest) kann über das Deployment-Ziel auf das **zuletzt stabile Docker-Image** zurückgerollt werden. Der Rollback-Prozess ist manuell, aber schnell ausführbar.